

Carbon-Aware Workflow Scheduler – Explanation

This document explains, in practical terms, how the CarbonSchedulerDemo algorithm works. The goal of the scheduler is to run a set of dependent tasks at times when the grid's carbon intensity is as low as possible, while still respecting task dependencies and the number of machines (nodes) available.

1. Inputs and Data Model

Each task in the workflow is represented by a Task object. The important fields are: - id: a unique integer for the task. - submissionTimeMs: when the task becomes available (in milliseconds). - durationMs: how long the task runs (in milliseconds). - parents: a list of task IDs that must finish before this task can start. - children: a list of task IDs that depend on this task (used mainly for clarity).

```
public static class Task {
    public final int id;
    public final long submissionTimeMs;
    public final long durationMs;
    public final int cpuCount;
    public final int cpuCapacity;
    public final int memCapacity;
    public final List<Integer> parents;
    public final List<Integer> children;

    public Task(int id,
                long submissionTimeMs,
                long durationMs,
                int cpuCount,
                int cpuCapacity,
                int memCapacity,
                List<Integer> parents,
                List<Integer> children) {
        this.id = id;
        this.submissionTimeMs = submissionTimeMs;
        this.durationMs = durationMs;
        this.cpuCount = cpuCount;
        this.cpuCapacity = cpuCapacity;
        this.memCapacity = memCapacity;
        this.parents = parents;
        this.children = children;
    }
}
```

The scheduler also needs: - carbonIntensity: an array where carbonIntensity[s] is the carbon per time slot s. - slotLengthMs: the real-world duration of a time slot (for example, one hour would be 3_600_000 ms). - numNodes: how many tasks can run in parallel in the same slot. The ScheduleResult simply reports, for each task, the chosen start slot and the total carbon of the schedule:

```
public static class ScheduleResult {
    public final Map<Integer, Integer> taskStartSlot; // taskId -> slot index
    public final double totalCarbon;

    public ScheduleResult(Map<Integer, Integer> taskStartSlot, double totalCarbon) {
        this.taskStartSlot = taskStartSlot;
        this.totalCarbon = totalCarbon;
    }
}
```

2. High-Level Idea

At a high level, the algorithm does this: 1. Build the dependency graph from the tasks (who depends on whom). 2. Topologically sort the tasks so that every task appears after its parents. 3. Convert each task's duration from milliseconds into a number of time slots. 4. Use depth-first search (DFS) to try all valid ways to place tasks into slots. 5. For each possible schedule, compute the total carbon emissions, taking into account partial-slot durations. 6. Use branch-and-bound to skip schedules that are already worse than the best one found so far. The final result is the schedule with the lowest total carbon that respects all dependencies and machine limits.

3. Building the Graph and Topological Order

First, the algorithm maps each task ID to an internal index so it can use arrays instead of hash maps. Then it builds a list of children and an in-degree counter (number of parents) for each task. This is used to compute a topological order using Kahn's algorithm: repeatedly take tasks with in-degree zero (no remaining unscheduled parents), add them to the order, and reduce the in-degree of their children.

```
// Build adjacency (children) and in-degrees
List<List<Integer>> children = new ArrayList<>();
for (int i = 0; i < n; i++) {
    children.add(new ArrayList<>());
}
int[] inDeg = new int[n];

for (Task t : tasks) {
    int idx = idToIndex.get(t.id);
    for (int parentId : t.parents) {
        Integer pIdx = idToIndex.get(parentId);
        children.get(pIdx).add(idx);
        inDeg[idx]++;
    }
}

// Kahn's algorithm for topological order
int[] topo = new int[n];
int topoCount = 0;
Deque<Integer> q = new ArrayDeque<>();
for (int i = 0; i < n; i++) {
    if (inDeg[i] == 0) q.add(i);
}

while (!q.isEmpty()) {
    int u = q.removeFirst();
    topo[topoCount++] = u;
    for (int v : children.get(u)) {
        if (--inDeg[v] == 0) q.add(v);
    }
}
```

The resulting topo array tells us the order in which to schedule tasks. Parents always appear before children, which means that when we consider a task, we already know where its parents were placed.

4. Time Slots and Task Durations

Time is split into fixed-size slots (for example, one slot per hour). Each task's duration in milliseconds is converted to a number of slots using a ceiling operation so that the task always has enough room to run. At the same time, we keep the exact duration in milliseconds so that carbon can be charged proportionally to how much of each slot the task actually uses.

```
int[] durationSlots = new int[n];
long[] durationMsArr = new long[n];
long minSubmission = Long.MAX_VALUE;
for (int i = 0; i < n; i++) {
```

```

Task t = tasks.get(i);
durationMsArr[i] = t.durationMs;

if (t.submissionTimeMs < minSubmission) {
    minSubmission = t.submissionTimeMs;
}
long dur = t.durationMs;
durationSlots[i] = (int) ((dur + slotLengthMs - 1) / slotLengthMs); // ceil
}

// Release slot based on submission time
int[] releaseSlot = new int[n];
for (int i = 0; i < n; i++) {
    Task t = tasks.get(i);
    long delta = t.submissionTimeMs - minSubmission;
    releaseSlot[i] = (delta <= 0) ? 0 : (int) ((delta + slotLengthMs - 1) / slotLengthMs);
}

```

5. Depth-First Search and Branch-and-Bound

The main search is performed by a recursive dfs method. It walks through the tasks in topological order and tries to assign each one to a start slot. For a given task, the algorithm: 1. Computes the earliest slot it can start in, based on its releaseSlot and the finish slots of its parents. 2. Iterates over all candidate start slots from earliest to the end of the horizon. 3. Checks that, in all the slots the task would occupy, there is free capacity ($\text{slotLoad}[s] < \text{numNodes}$). If the placement is feasible, it computes the carbon cost for this task using the exact duration in milliseconds and the carbon intensity of each slot it touches.

```

private static void dfs(int index,
                        int[] topo,
                        int[][] parents,
                        int[] durationSlots,
                        long[] durationMsArr,
                        int[] releaseSlot,
                        double[] carbon,
                        long slotLengthMs,
                        int numNodes,
                        int[] slotLoad,
                        int[] assignment,
                        double currentCost,
                        double[] bestCostBox,
                        int[] bestAssignment) {

    int n = topo.length;
    int H = carbon.length;

    if (index == n) {
        if (currentCost < bestCostBox[0]) {
            bestCostBox[0] = currentCost;
            System.arraycopy(assignment, 0, bestAssignment, 0, n);
        }
        return;
    }

    int taskIdx = topo[index];
    int durSlots = durationSlots[taskIdx];
    long durMs = durationMsArr[taskIdx];

    // earliest start
    int earliest = releaseSlot[taskIdx];
    for (int pIdx : parents[taskIdx]) {
        int parentStart = assignment[pIdx];
        int parentFinish = parentStart + durationSlots[pIdx];
        if (parentFinish > earliest) {
            earliest = parentFinish;
        }
    }
    for (int h = earliest; h < H; h++) {

```

```

    int end = h + durSlots;
    if (end > H) break;

    boolean fits = true;
    for (int s = h; s < end; s++) {
        if (slotLoad[s] >= numNodes) {
            fits = false;
            break;
        }
    }
    if (!fits) continue;

    // fractional carbon cost
    double taskCost = 0.0;
    long remaining = durMs;
    for (int k = 0; k < durSlots && remaining > 0; k++) {
        int slot = h + k;
        long usedInThisSlot = Math.min(remaining, slotLengthMs);
        double fraction = (double) usedInThisSlot / (double) slotLengthMs;
        taskCost += carbon[slot] * fraction;
        remaining -= usedInThisSlot;
    }

    double newCost = currentCost + taskCost;
    if (newCost >= bestCostBox[0]) continue; // branch-and-bound

    assignment[taskIdx] = h;
    for (int s = h; s < end; s++) slotLoad[s]++;

    dfs(index + 1, topo, parents, durationSlots, durationMsArr, releaseSlot,
        carbon, slotLengthMs, numNodes,
        slotLoad, assignment, newCost, bestCostBox, bestAssignment);

    for (int s = h; s < end; s++) slotLoad[s]--;
    assignment[taskIdx] = -1;
}
}

```

The branch-and-bound part is the line that compares `newCost` with `bestCostBox[0]`. If the partial cost is already worse than the best full schedule seen so far, there is no point exploring that branch further, so it is skipped. This keeps the search tractable while still guaranteeing that the final answer is the lowest-carbon schedule the model can represent.

6. Example Usage

Here is a small example of how you might call the scheduler. Tasks are defined with durations in milliseconds, a number of nodes is available, and carbon intensity values come from some external forecast.

```

public static void main(String[] args) {
    long start_time_ms = 0L;

    List<Task> tasks = Arrays.asList(
        new Task(0, start_time_ms, 900_000L, 1, 3_000, 10_000,
            List.of(), List.of(1)),
        new Task(1, start_time_ms, 1_800_000L, 1, 3_000, 10_000,
            List.of(0), List.of(2))
        // ... more tasks ...
    );

    double[] carbon = { 321.37, 320.29, 304.04, /* ... */ };

    long slotLengthMs = 3_600_000L; // 1 hour
    int numNodes = 1;

    ScheduleResult result = schedule(tasks, numNodes, carbon, slotLengthMs);
}

```

```
System.out.println("Total carbon: " + result.totalCarbon);
for (Map.Entry<Integer, Integer> e : result.taskStartSlot.entrySet()) {
    System.out.println("Task " + e.getKey() + " starts in slot " + e.getValue());
}
}
```

7. Summary

In short, this algorithm takes a DAG of tasks, time-varying carbon data, and a fixed number of machines, and searches for the schedule that minimizes total carbon emissions. It respects dependencies and machine limits, supports real durations (in ms), and charges carbon proportionally to how long each task runs within each time slot. Depth-first search explores possible schedules, and branch-and-bound keeps the search efficient by cutting off obviously worse options early.